

Programming Techniques in Java

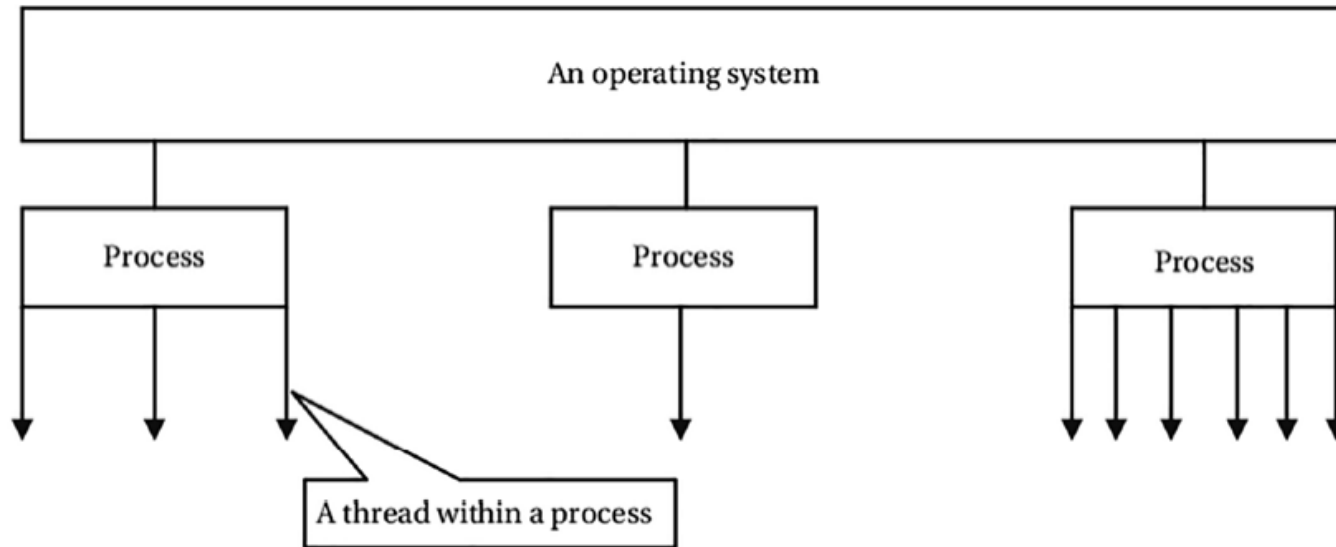
Programming Techniques with Threads

Main bibliographic sources

- <https://docs.oracle.com/javase/tutorial/essential/concurrency/>
- <https://jenkov.com/tutorials/java-concurrency/index.html>
- Brian Goetz, Tim Peierls, Joshua Bloch, Joseph Bowbeer, David Holmes, and Doug Lea, Java Concurrency in Practice, Addison Wesley, Pearson Education
- K. Sharan, Beginning Java 8 Language Features: Lambda Expressions, Inner Classes, Threads, I/O, Collections, and Streams 1st Edition, APRESS, 2014.

T. Cioara, V. Chifu, C. Pop, D. Mitrea
2026

Processes and Threads



Process = address space + resources + threads

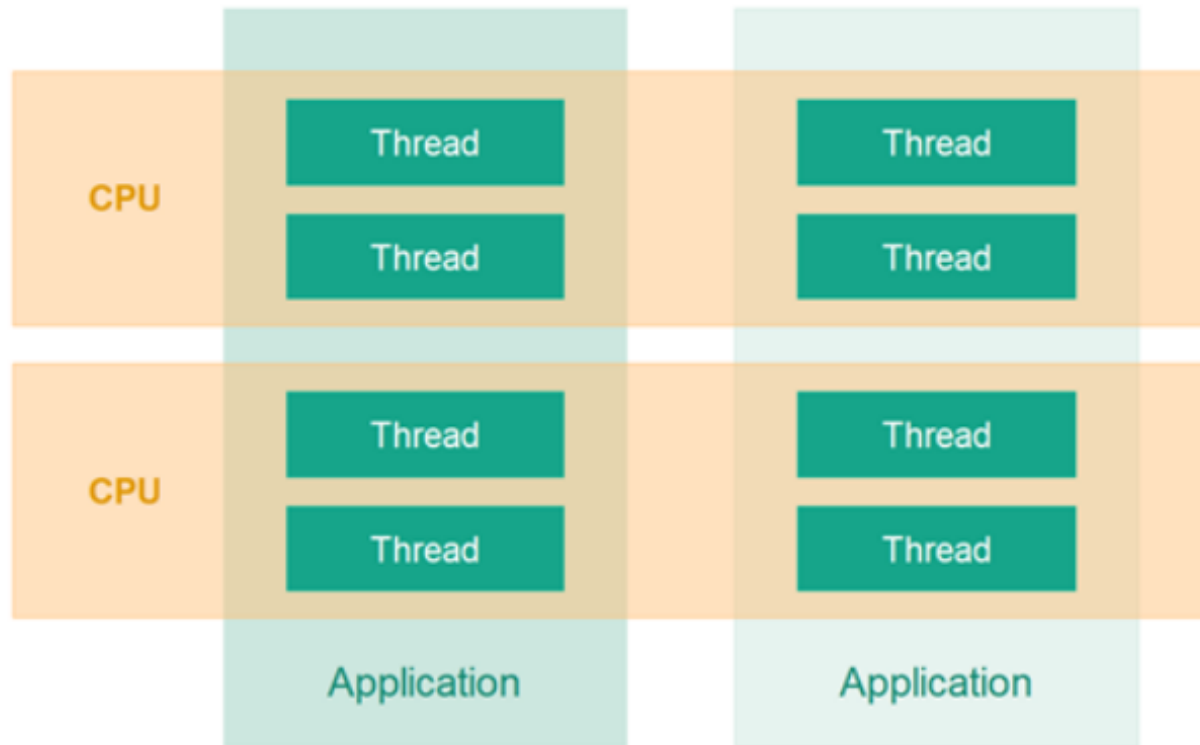
- Running instance of a program
- Communicates with other processes using Inter Process Communication (IPC) resources, such as pipes and sockets

Thread = Unit of execution within a process

- Shared access to address space and resources of the process
- Maintains a program counter, a stack and a private memory
- Communicate with each others

Multithreading

- Having multiple threads of execution inside the same application
- A thread is like a separate CPU executing your application



Multithreading

- Benefits
 - Better utilization of a single CPU.
 - Better user experience with regards to responsiveness.
 - Better user experience with regards to fairness.
- Costs
 - More complex design
 - Context Switching Overhead
 - Increased Resource Consumption

Multitasking vs Multithreading

- **Multitasking**

- Computers can execute multiple programs “at the same time”
 - The CPU is shared between programs
 - The operating system switches between the running programs, executing each of them for a while before switching

- **Multithreading**

- Multiple threads of execution inside the same program
- A thread of execution is similar to a CPU executing the program
- More challenging than multitasking
 - The threads are executing in the same program and are reading and writing the same memory simultaneously possibly resulting in errors
 - Not applicable to single CPU machines – two threads never execute simultaneously
 - Applicable to machines with multi core CPUs/multiple CPUs

Creating a Thread in Java

Directly control thread creation and management

Instantiate Thread each time the application needs to initiate an asynchronous task

OR

Abstract Thread management

Pass the application's tasks to an executor

Creating a Thread in Java

1. Inheriting your class from the Thread Class

```
public class MyThreadClass extends Thread {
    @Override
    public void run() {
        System.out.println("Hello Java thread!");
    }
    // More code goes here
}
...
MyThreadClass myThread = new MyThreadClass();
myThread.start();
```

2. Implementing the Runnable Interface

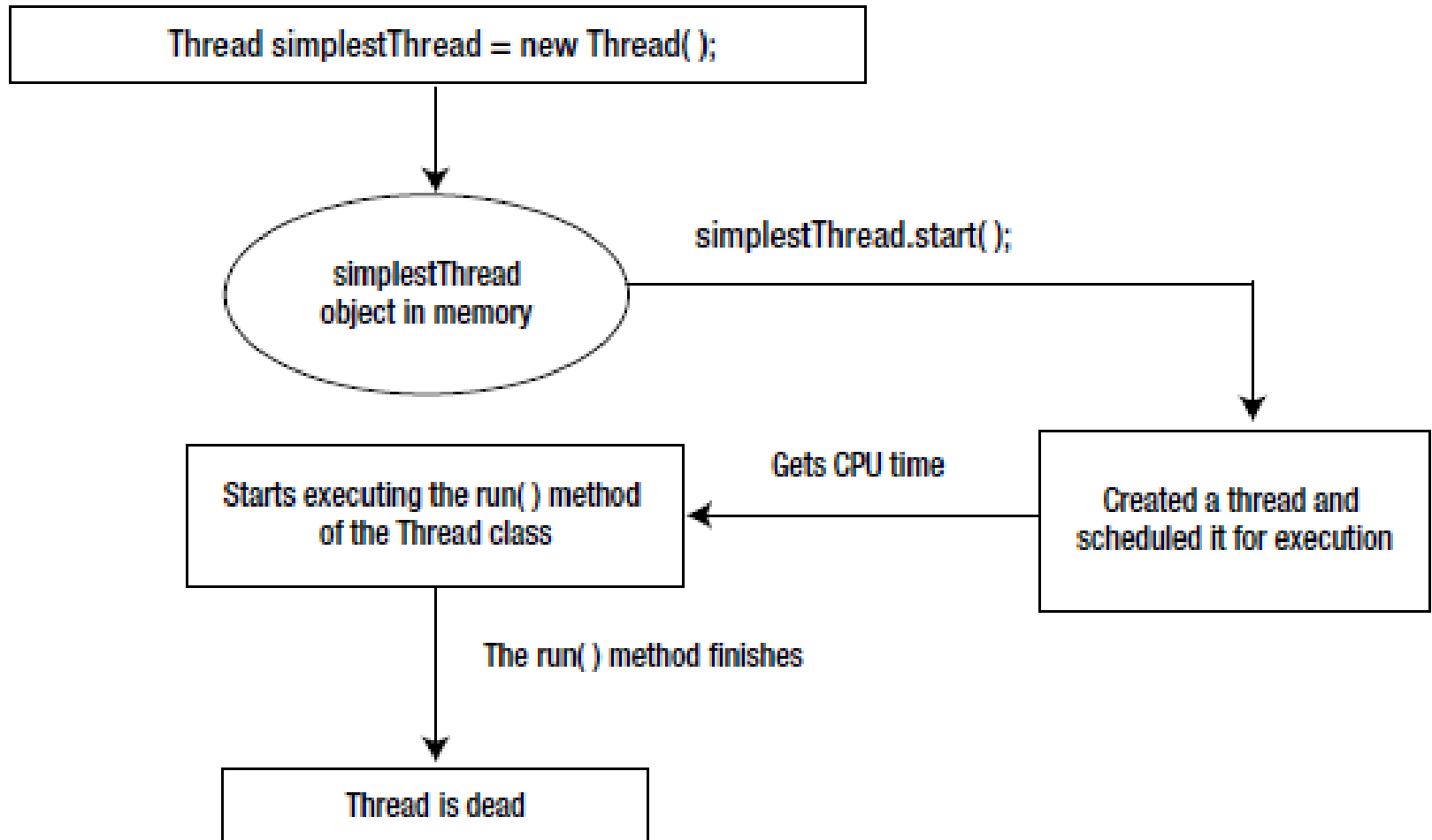
```
@FunctionalInterface
public interface Runnable
{
    void run();
}
```

```
public class RunnableClass implements Runnable{
    public void run(){
        System.out.println("class impl. Runnable");
    }
}
...
Thread myThread = new Thread(new RunnableClass());
myThread.start();
```

Thread versus Runnable

Criteria	Features	
Inheritance	Extend Thread	Cannot extend another class
	Implement Runnable	Can extend another class and can implement other interfaces
Reusability	Extend Thread	Contains both thread and job specific behavior code
	Implement Runnable	Contains only the functionality we want in the run method
Classes vs interfaces	Extend Thread	Defines the core identity of the new class
	Implement Runnable	Describes some abilities of the new class
Coupling	Extend Thread	Tight coupling
	Implement Runnable	Loose coupling – the code is split in 2 parts: behavior and thread
Overhead	Extend Thread	Additional overhead because of inheritance
	Implement Runnable	-

Creating a Thread in Java



Thread class methods

- Purpose: Control Thread Execution
- Thread represents a running unit of execution
- Methods here control thread lifecycle and scheduling
- They affect the current thread itself
- No object monitor involvement
- Examples
 - `sleep()` → Pause current thread
 - `join()` → Wait for another thread to finish
 - `yield()` → Hint scheduler
 - `interrupt()` → Signal interruption

Pause a Thread

- **Thread.sleep()**
 - For a specified duration
 - The current thread will be put in the wait state

Exception thrown in case the current thread is interrupted by another thread while *sleep* is active.

```
public class App{
    public static void printMessages() throws InterruptedException{
        for (int i=0; i< 10; i++){
            System.out.println( "Sending message number " + i);
            Thread.sleep(4000);
        }
    }

    public static void main( String[] args ) throws InterruptedException{
        App.printMessages();
    }
}
```

The time for suspending the execution of a running thread must be given in milliseconds and must be a positive number.

Methods in the Object Class

- **Purpose: Inter-Thread Coordination**
- Every object in Java has a **monitor (intrinsic lock)**
- Threads synchronize **on objects**
- Waiting happens **on an object's monitor**
- Communication happens between threads via shared objects
- **Methods**
 - `wait()` → Release object lock & wait
 - `notify()` → Wake one waiting thread
 - `notifyAll()` → Wake all waiting threads

Example: wait() vs sleep ()

- **wait()** – makes a thread to wait to manage synchronization
 - The thread releases the lock associated to the object and wait
 - wake up -> use **notifyAll()** or **notify()** or waiting time has expired

Wait()	Sleep()
Wait() method belongs to Object class.	Sleep() method belongs to Thread class.
Wait() method releases lock during Synchronization.	Sleep() method does not release the lock on object during Synchronization.
Wait() should be called only from Synchronized context.	There is no need to call sleep() from Synchronized context.
Wait() is not a static method.	Sleep() is a static method.

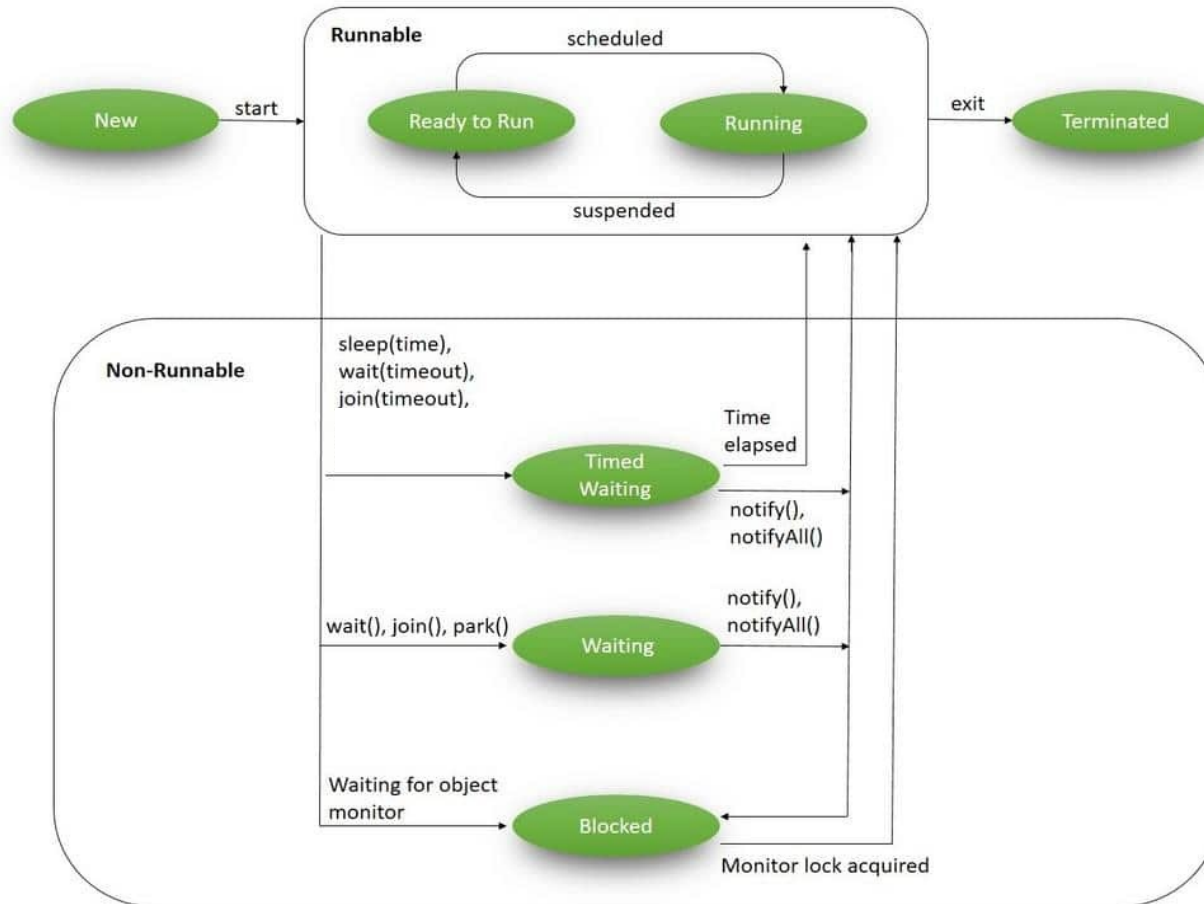
Comparison

Concern	Thread	Object
Controls execution timing	✓	✗
Coordinates shared access	✗	✓
Works with synchronization	✗	✓
Affects current thread only	✓	✗

Thread → Execution control

Object → Coordination & communication

Lifecycle of a Thread



Threads Issues

```
public class Buffer {  
    private int number = -1;  
    public int getNumber() { return number; }  
    public void setNumber(int number) { this.number = number; }  
}
```

Shared resource

The threads

```
public class Producer extends Thread{  
    private Buffer buffer;  
    public Producer(Buffer buffer){  
        this.buffer = buffer;  
    }  
    public void run(){  
        for(int i=0; i < 10; i++){  
            buffer.setNumber(i);  
            System.out.println("Producer set:"+i);  
            try{  
                sleep(1000);  
            }catch (InterruptedException e) {  
                e.printStackTrace();  
            }  
        }  
    }  
}
```

```
public class Consumer extends Thread{  
    private Buffer buffer;  
    public Consumer(Buffer buffer){  
        this.buffer = buffer;  
    }  
    public void run(){  
        int value = 0;  
        for(int i=0; i<10; i++){  
            value = buffer.getNumber();  
            System.out.println("Consumer  
received:"+value);  
        }  
    }  
}
```

```
Producer set:0  
Consumer received:0  
Consumer received:0  
Consumer received:0  
Consumer received:0  
Consumer received:0  
Consumer received:0  
Consumer received:0  
Consumer received:0  
Consumer received:0  
Consumer received:0  
Consumer received:0  
Consumer received:0  
Producer set:1  
Producer set:2  
Producer set:3  
Producer set:4  
Producer set:5  
Producer set:6  
Producer set:7  
Producer set:8  
Producer set:9
```

**Not the expected
result!**

[Source](#)

Critical Sections and Race Conditions

- **Critical section**
 - Section of code executed by multiple threads and the sequence of execution for the threads makes a difference in the result of the concurrent execution of the critical section
 - When the result of multiple threads executing a critical section may differ depending on the sequence in which the threads execute => the critical section contains a **race condition**
- **Types of race conditions**
 - Read-modify-write
 - Check-then-act

Critical Sections and Race Conditions

- **Read-Modify-Write Race Condition**

- Two or more threads first read a given variable, then modify its value and write it back to the variable
 - The new value must depend one way or another on the previous value.
 - **Problem that can occur:** if two threads read the value from Heap memory into their local memory (mapped on CPU registers) then modify the value and then write the values back

```
public class Counter {  
    protected long count = 0;  
    public void add(long value){  
        this.count = this.count + value;  
    }  
}
```

Critical section

may fail if executed by multiple threads simultaneously

this.count = 0;

A: Reads this.count into a register (0)
B: Reads this.count into a register (0)
B: Adds value 2 to register
B: Writes register value (2) back to memory.
 this.count now equals 2
A: Adds value 3 to register
A: Writes register value (3) back to memory.
 this.count now equals 3

Critical Sections and Race Conditions

- **Check-then-act Race Condition**

- Occurs when two threads check the same condition, then act upon that condition in a way that changes the condition
- If two threads both check the condition at the same time, and then one thread goes ahead and changes the condition, this can lead to the other thread acting incorrectly on that condition
- Example

```
public class CheckThenActExample {  
    public void checkThenAct(Map<String, String> sharedMap) {  
        if(sharedMap.containsKey("key")){  
            String val = sharedMap.remove("key");  
            if(val == null) {  
                System.out.println("Value for 'key' was null");  
            }  
        } else {  
            sharedMap.put("key", "value");  
        }  
    }  
}
```

Preventing Race Conditions

- Execute a critical section as an **atomic instruction**
 - Once a single thread is executing the critical section no other threads can execute it until the first thread has left the critical section
- Race conditions can be avoided by **proper thread synchronization in critical sections**
- Thread synchronization can be achieved using:
 - a synchronized block of Java code
 - other synchronization constructs like locks or atomic variables like `java.util.concurrent.atomic.AtomicInteger`.

Preventing Race Conditions

- **Locking - synchronized statements**

- To preserve state consistency, update state variables in a single atomic operation!

**Block of code to
be guarded by
the lock**

```
synchronized (lock) {//Reference to an object  
    // Access or modify shared state guarded by lock  
    ...  
}
```

- Every Java object can implicitly act as a lock for purposes of synchronization
 - ***intrinsic locks***
 - Automatically acquired and released
 - A happens-before relationship is established

Preventing Race Conditions

- **Synchronized blocks**

```
public class TwoSums {  
  
    private int sum1 = 0;  
    private int sum2 = 0;  
  
    public void add(int val1, int val2){  
        synchronized(this){  
            this.sum1 += val1;  
            this.sum2 += val2;  
        }  
    }  
}
```

or

```
public class TwoSums {  
  
    private int sum1 = 0;  
    private int sum2 = 0;  
  
    private Integer sum1Lock = new Integer(1);  
    private Integer sum2Lock = new Integer(2);  
  
    public void add(int val1, int val2){  
        synchronized(this.sum1Lock){  
            this.sum1 += val1;  
        }  
        synchronized(this.sum2Lock){  
            this.sum2 += val2;  
        }  
    }  
}
```

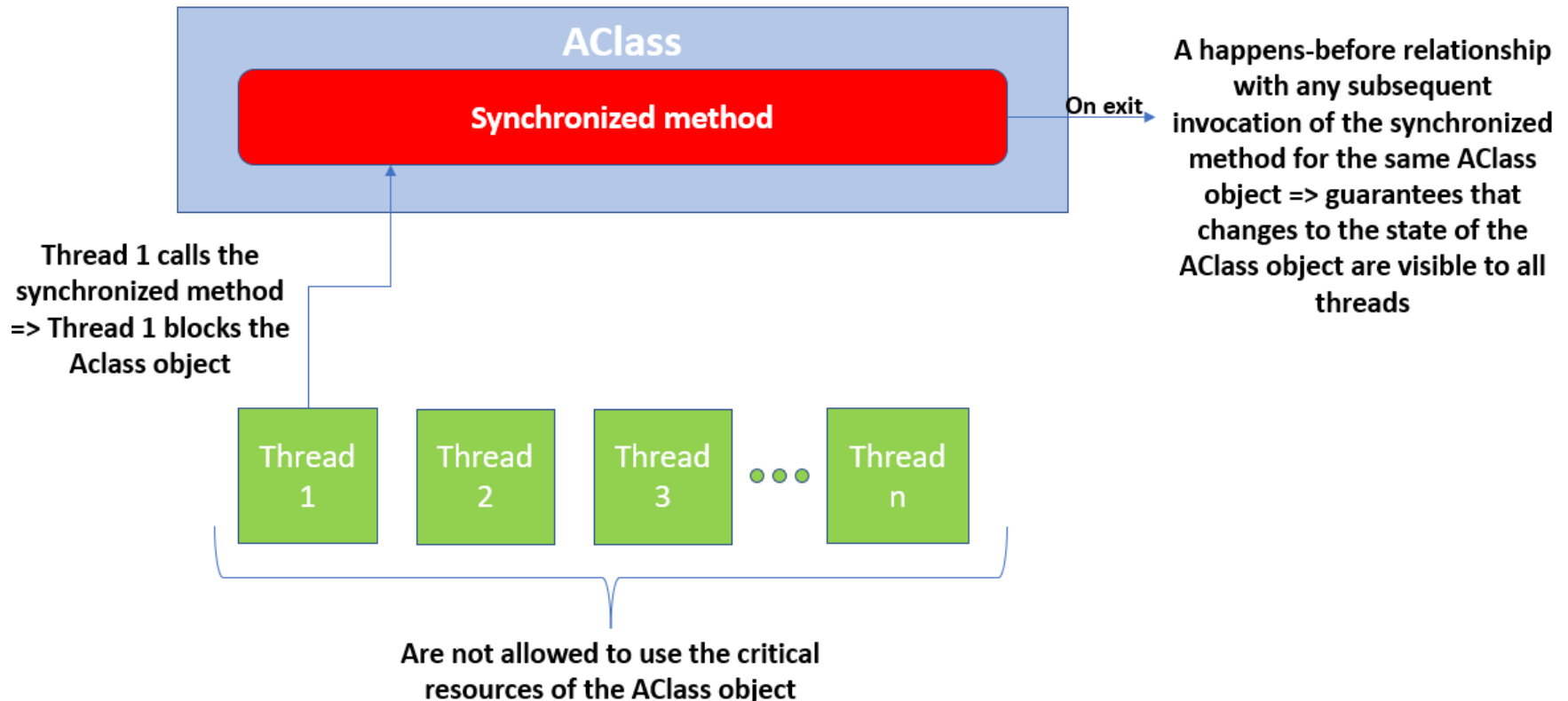
The two sum variables are independent of each other so their summing up can be split into two separate synchronized blocks

Two threads can execute the add method at the same time.

Preventing Race Conditions

- **Synchronized Methods**

- **synchronized** keyword in the methods' declaration
- **Constructors cannot be synchronized**



Preventing Race Conditions

- **Atomicity**

- Atomic action cannot be interleaved => avoids thread interference
- java.util.concurrent.atomic package
- Reads and writes are atomic for reference variables and for most primitive variables

Compound Operations

```
int i=0;
i++; // Get I value & add one to it
/*Accessed simultaneously by both Th1 and Th2
Can lead to inconsistencies:
- result can be 1(both threads got 0 and
  incremented to 1)
- result can be 2(second thread got the value
  1 incremented by the first thread)
*/
```

Atomic Operations

```
AtomicInteger i= new AtomicInteger();
i.getAndIncrement ();
```

i++ is not atomic!

- *read-modify-write* operation
- not stateless and is not thread-safe due to instance variable

Race conditions

Thread safety and Shared Resources

- **Thread safe code** = code that can be executed simultaneously by multiple threads
 - It contains no race conditions
- Race condition only occurs when multiple threads update shared resources
- **Local variables**
 - stored in each thread's own stack
 - are never shared between threads => all local primitive variables are thread safe

```
public void someMethod(){  
    long threadSafeInt = 0;  
    threadSafeInt++;  
}
```

Example of thread-safe code

Thread safety and Shared Resources

- **Local object references**

- The reference is not shared
- The object referenced is stored in the shared heap
- An object is thread safe if
 - It is created locally and never escapes the method it was created in
 - If it is passed to other methods and objects and these methods or objects do not make the passed object available to other threads

```
public void someMethod(){  
  
    LocalObject localObject = new LocalObject();  
  
    localObject.callMethod();  
    method2(localObject);  
}  
  
public void method2(LocalObject localObject){  
    localObject.setValue("value");  
}
```

Thread safety and Shared Resources

- **Object member variables (fields)**
 - are stored on the heap along with the object
 - if two threads call a method on the same object instance and this method updates object member variables, the method is not thread safe

```
public class NotThreadSafe{  
    StringBuilder builder = new StringBuilder();  
    public add(String text){ this.builder.append(text);}  
}
```

← **The method is not thread safe**

```
public class MyRunnable implements Runnable{  
    NotThreadSafe instance = null;  
    public MyRunnable(NotThreadSafe instance){  
        this.instance = instance;  
    }  
    public void run(){  
        this.instance.add("some text");  
    }  
}
```

Race Condition

```
NotThreadSafe sharedInstance = new NotThreadSafe();  
new Thread(new MyRunnable(sharedInstance)).start();  
new Thread(new MyRunnable(sharedInstance)).start();
```

Thread Safety and Shared Resources

- Example - Synchronized Methods and Threads Coordination
A Solution for the Producer-Consumer Problem

```
public class Buffer {
    private int number = -1;
    private boolean available = false;
    public synchronized int get(){
        while(!available){
            try{ wait(); }catch(InterruptedException ex){
                ex.printStackTrace();
            }
        }
        available = false;
        notifyAll();
        return number;
    }
    public synchronized void put(int number){
        while(available){
            try{ wait(); }catch(InterruptedException ex){
                ex.printStackTrace();
            }
        }
        this.number = number;
        available = true;
        notifyAll();
    }
}
```

Critical section

Critical section

UTCN - Programming Techniques

Source: C. Frasinaru,
Curs practic de Java,
Matrix ROM.

Expected
result

```
Consumer received:0
Producer set:0
Producer set:1
Consumer received:1
Producer set:2
Consumer received:2
Producer set:3
Consumer received:3
Producer set:4
Consumer received:4
Consumer received:5
Producer set:5
Producer set:6
Consumer received:6
Consumer received:7
Producer set:7
Producer set:8
Consumer received:8
Producer set:9
Consumer received:9
```

Thread safety and Immutability

- If multiple threads read the same resource race conditions do not occur
- If objects shared by multiple threads are immutable then they are thread safe

```
public class Employee{  
    private final String employeeID;  
    private final String firstName;  
    private final String lastName;  
  
    //constructor-assigns values to all fields  
    public Employee(String id, String first, String last){  
        employeeID = id;  
        firstName = first;  
        lastName = last;  
    }  
    public int getEmployeeID() { return Integer.parseInt(employeeID); }  
    }  
    // should be removed  
    public void setEmployeeID(int id) { employeeID = Integer.toString(id); }  
    ...  
}
```

Example of immutable class

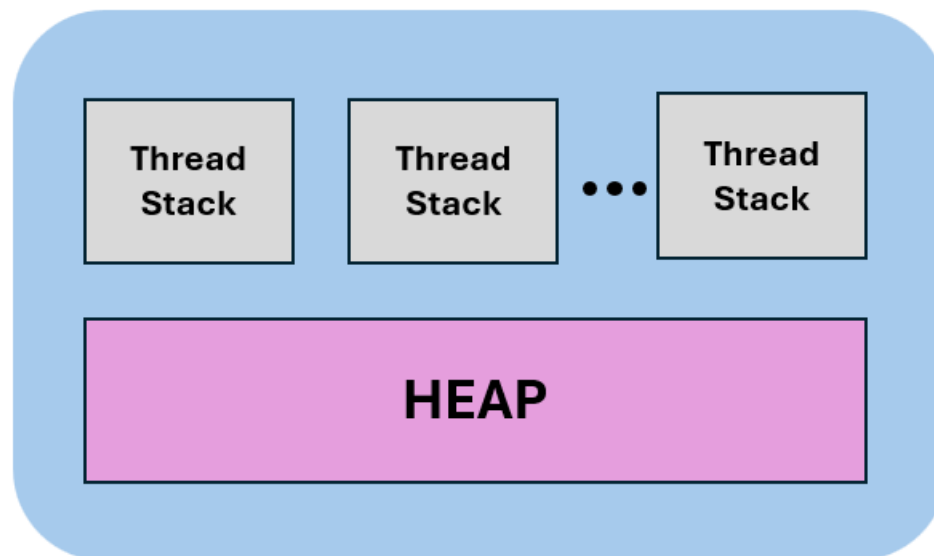
Thread Safety and Stateless Objects

- **Stateless objects** are always thread-safe!

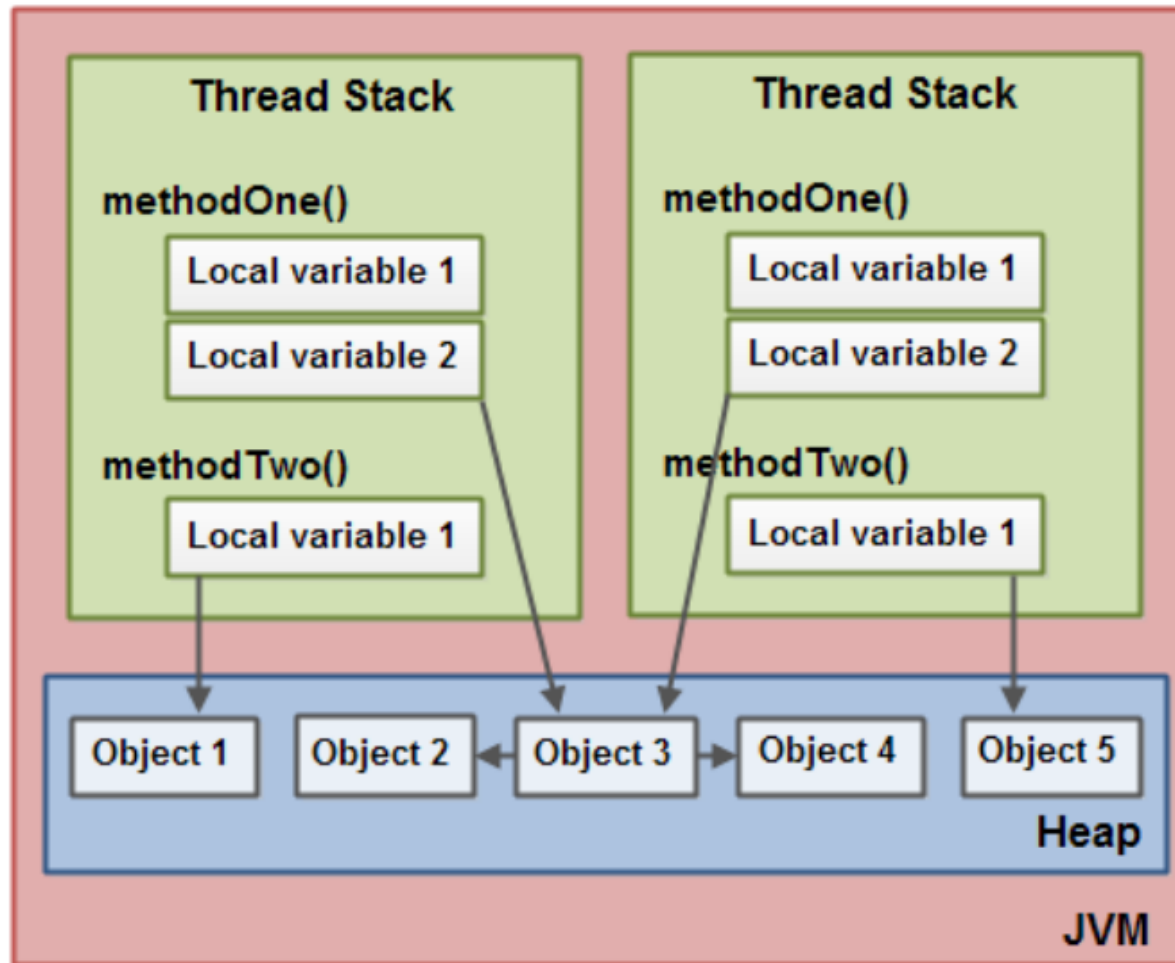
```
public class TransactionManager {  
    public double executeTransaction(Transaction transaction){  
        ...  
    }  
}
```

Java Memory Model

- Specifies how the Java Virtual Machine (JVM) works with the computer's memory (RAM)
- Memory model of the JVM
- Specifies
 - How and when different threads can see values written to shared variables by other threads
 - How to synchronize access to shared variables when necessary



Java Memory Model



Java Memory Model

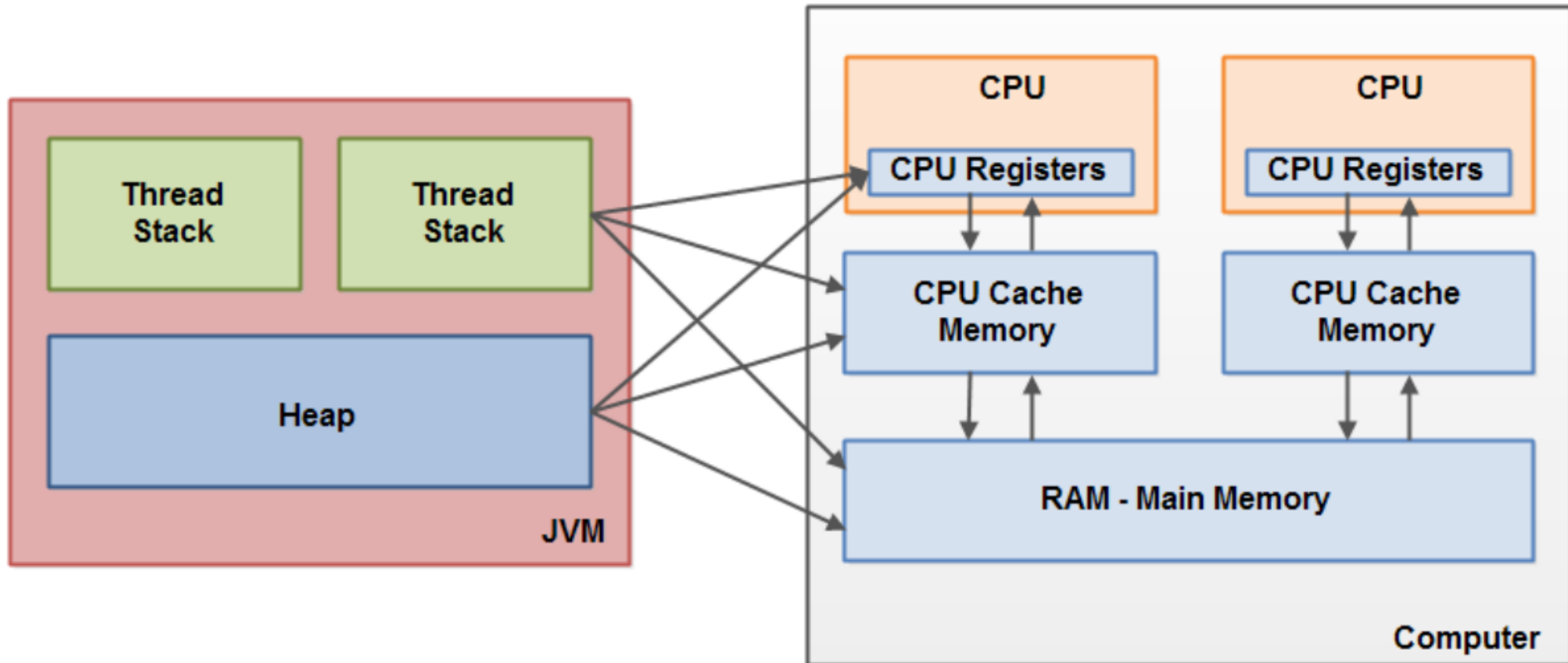
- **Thread stack (Call stack)**
 - Contains
 - Information about what methods the thread has called to reach the current point of execution
 - All local variables for each method being executed
 - A thread can access only its own thread stack
 - Local variables created by a thread are invisible to all other threads than the thread who created it
 - Each thread has its own version of each local variable
 - All local variables of primitive types (boolean, byte, short, char, int, long, float, double) are fully stored on the thread stack => not visible to other threads

Java Memory Model

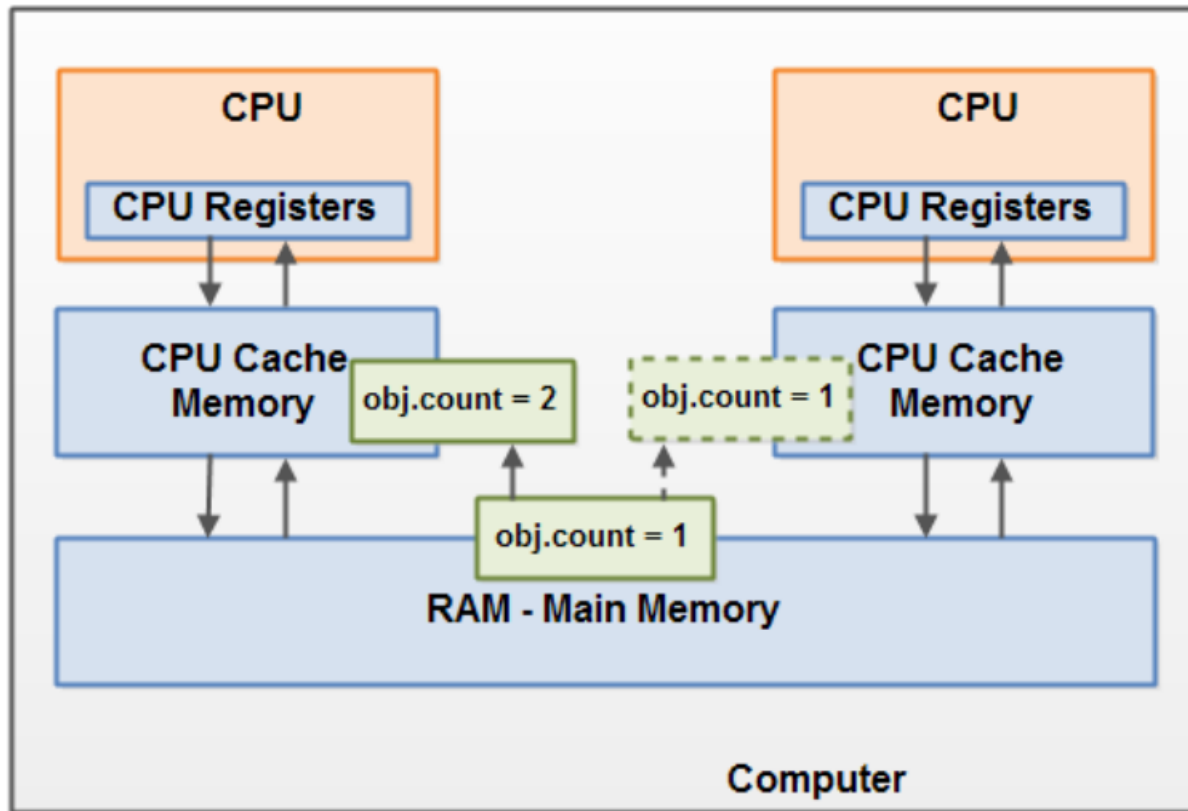
- **Heap**

- Contains all objects created in a Java application, regardless of what thread created the object
 - Includes the object versions of the primitive types (e.g., Byte, Integer, Long etc.)
 - If a local variable is a reference to an object, then the reference is stored on the thread stack, but the object is stored on the heap.
 - An object's member variables are stored on the heap along with the object itself (for member variables of primitive type and reference type)
 - Static class variables are also stored on the heap along with the class definition.
- Objects on the heap can be accessed by all threads that have a reference to the object
- When a thread has access to an object, it can also get access to that object's member variables
- If two threads call a method on the same object at the same time, they will both have access to the object's member variables, but each thread will have its own copy of the local variables.

Java Memory Model and the Hardware Memory Architecture

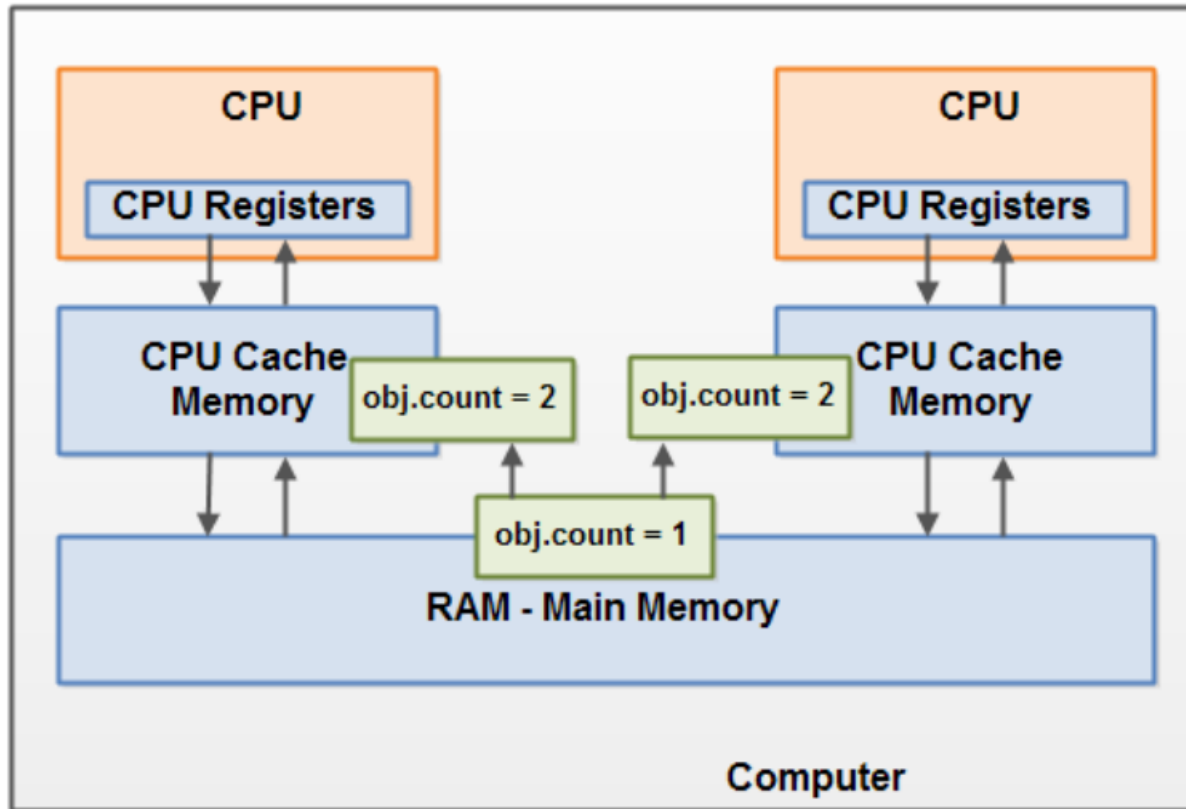


Java Memory Model - Visibility of Shared Objects



PROBLEM: A thread on the left CPU updates the shared object's count to 2 in its cache, but this change isn't yet visible to threads on the right CPU because it hasn't been written back to main memory.

Java Memory Model – Race Conditions



PROBLEM: CONCURRENT INCREMENT WITHOUT PROPER SYNCHRONIZATION!

SOLUTION: Use a Java synchronized block to ensure only one thread enters a critical section at a time, while keeping shared variables consistent by syncing with main memory.

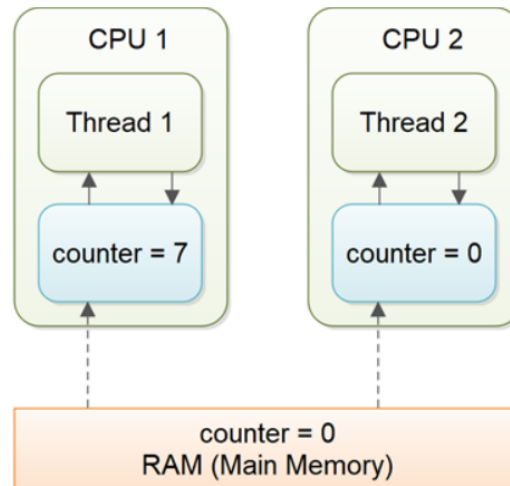
Java Volatile Keyword

- Used to mark a Java variable as "being stored in main memory".
- Every read of a volatile variable will be read from the computer's main memory, and not from the CPU registers
- Every write to a volatile variable will be written to main memory, and not just to the CPU registers

Java Volatile Keyword

- Variable Visibility Problems
 - With non-volatile variables there are no guarantees about when the Java Virtual Machine (JVM) reads data from main memory into CPU registers, or writes data from CPU registers to main memory
 - Example
 - SharedObject instance shared by 2 threads
 - Thread 1 increments the counter variable
 - Threads 1 and 2 read the counter variable

```
public class SharedObject {  
    public int counter = 0;  
}
```



If the counter variable is not declared volatile there is no guarantee about when the value of the counter variable is written from the CPU registers back to main memory

Java Volatile Keyword

- Variable Visibility Problems
 - By declaring the counter variable **volatile** all writes to the counter variable will be written back to main memory immediately
 - All reads of the counter variable will be read directly from main memory.

```
public class SharedObject {  
    public volatile int counter = 0;  
}
```

If thread T1 modifies the counter, and thread T2 reads the counter (but never modifies it), declaring the counter variable volatile is enough to guarantee visibility for T2 of writes to the counter variable.

If, both threads T1 and T2 were incrementing the counter variable, then declaring the counter variable volatile would not have been enough!

Java Volatile Keyword

- When **volatile** Works
 - One thread writes, others read 
 - Multiple threads write independent values 
- When **volatile** Fails
 - When operation depends on previous value 
 - Why This Happens
 - Read → Modify → Write is not atomic
 - Gap between read and write allows interference
 - Takeway
 - volatile ≠ thread-safe for compound operations
 - Use: synchronized, atomic operations, atomic data types

Threads Issues

- **Liveness problems - Deadlock**

- Two or more threads are blocked forever
- Each thread is waiting for a lock held by another thread

```
public void transfer(Account a,
                    Account b, double sum){
    synchronized(a){// Th1 locks acc. a
        //Th2 locks account b
        synchronized(b){
            //transfer sum
        }
    }
}
...
Call function :
Th1: transfer(a,b,sum1);
Th2: transfer(b,a,sum1);
```

Deadlock situation!!!

- How to prevent: lock ordering, timeouts, avoid nested locks

Threads Issues

- **Liveness problems - Starvation**

- A thread is starved when it never gets CPU time or resources
- A thread is always ready to run, but never executes
- Causes:
 - High-priority threads dominate CPU, while low-priority threads get little or no execution time
 - Threads wait to enter a synchronized block but some threads repeatedly win access
 - Multiple threads wait on same object but notify() wakes only one thread
- How to prevent: use locks instead of synchronized blocks, prefer notifyAll() over notify()

Threads Issues

- **Thread interference**

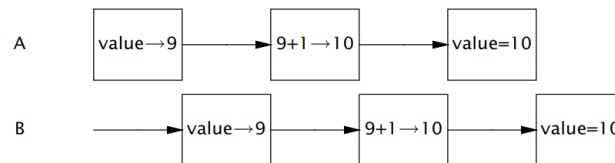
- Occurs when multiple threads modify shared data
- Results in unexpected or incorrect values

```
@NotThreadSafe
public class UnsafeSequence {
    private int value;

    /** Returns a unique value. */
    public int getNext() {
        return value++;
    }
}
```



LISTING 1.1. Non-thread-safe sequence generator.



- How to prevent
 - Use synchronized blocks
 - Use atomic classes (e.g., AtomicInteger)
 - Use locks (ReentrantLock)

Threads Issues

- **Memory inconsistency**
 - Threads see different values of the same variable
 - Due to CPU caching / delayed updates
- How to prevent
 - Use volatile → ensures visibility
 - Use synchronized → ensures consistency + ordering

Concurrency Utilities

- **Thread Safe Collections – Synchronized Collections**
 - Synchronization wrappers which create synchronized views of collections
 - *synchronizedCollection*, *synchronizedList*, *synchronizedMap*, etc.

```
List<String> list = Collections.synchronizedList(new ArrayList<String>());
```

For iteration, the collection needs to use external sync

- Achieve thread-safety through intrinsic locks
- Synchronized collections are thread safe

Must manually
synchronize on
the returned
collection when
iterating over it

```
Collection c = Collections.synchronizedCollection(myCollection);  
...  
synchronized (c) {  
    Iterator i = c.iterator(); // Must be in the  
                               // synchronized block  
    while (i.hasNext()) foo(i.next());  
}
```

Concurrency Utilities

- **Thread Safe Collections – Concurrent Collections**
 - Designed for concurrent accesses from multiple threads
 - *java.util.concurrent package: `BlockingQueue`, `ConcurrentHashMap`, `ConcurrentNavigableMap`, `CopyOnWriteArrayList`*
 - Achieve thread-safety
 - **BlockingQueue** – provides blocking put and take methods
 - Support the producer-consumer design patterns
 - **ConcurrentHashMap** - divides its data into segments
 - Different threads can acquire locks on each segment
 - Multiple threads can access the map at the same time
 - **CopyOnWriteArrayList** - creates a separate copy of List for each write operation

Are much more performant than synchronized collections

Executor Framework

- **Executors and Interfaces**

- Separates thread management and creation from the rest of application
- Executors - objects encapsulating thread management and creation
- *java.util.concurrent* package interfaces
 - **Executor** – supports launching new tasks – method **execute**
 - **ExecutorService** - adds method **submit** on Callable objects
 - **ScheduledExecutorService** - supports future and/or periodic execution of tasks

```
Executor exec= Executors.newFixedThreadPool(100);
Runnable task = new Runnable() {
    public void run() {
        //execute job
    }
};
exec.execute(task);
```

you can replace
new Thread(r)).start();
With: e.execute(r);

- A Callable does return a result and can throw a checked exception.
- Future object

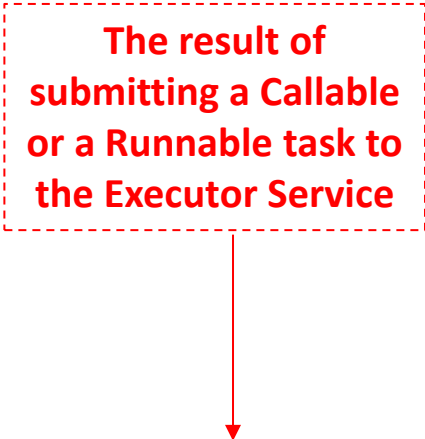
Executor Framework

- **Result Bearing Jobs**

- **Runnable** – suitable when we are not looking for a thread execution result
- **Callable** – suitable when we are looking for a thread execution result

```
public class FactorialTask implements Callable<Integer>{
    int number;
    public Integer call() throws InvalidParamaterException{
        int fact = 1;
        // ...
        for(int count = number; count > 1; count--){
            fact = fact * count;
        }
        return fact;
    }
}
```

**The result of
submitting a Callable
or a Runnable task to
the Executor Service**

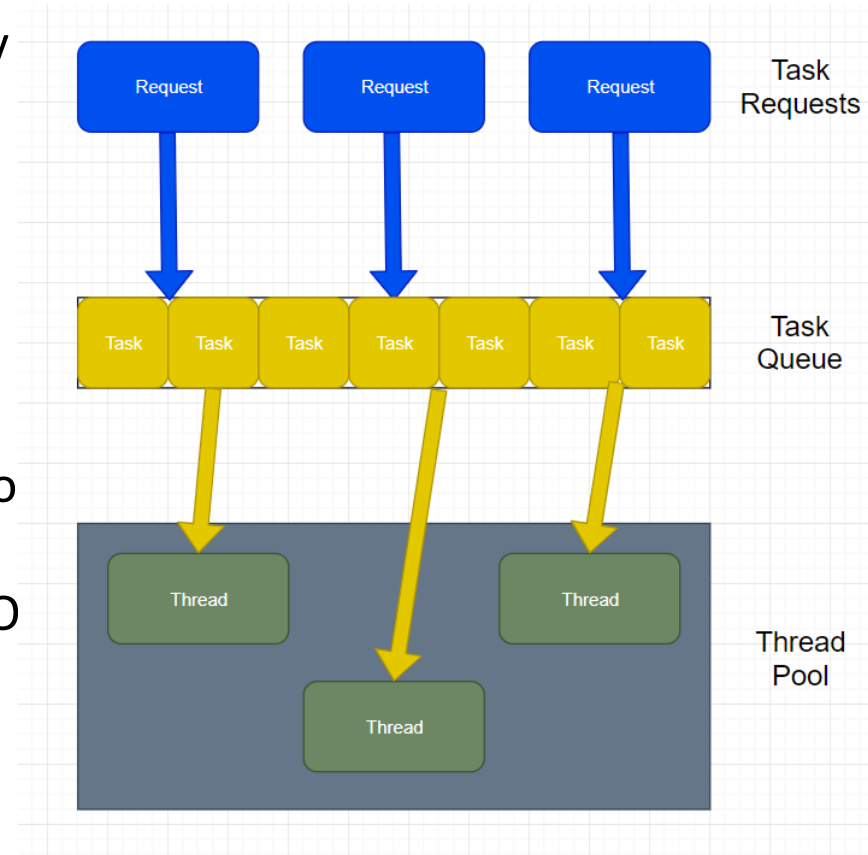


```
public void whenTaskSubmitted_ThenFutureResultObtained(){
    FactorialTask task = new FactorialTask(5);
    Future<Integer> future = executorService.submit(task);
    assertEquals(120, future.get().intValue());
}
```

Executor Framework

- **Thread Pools**

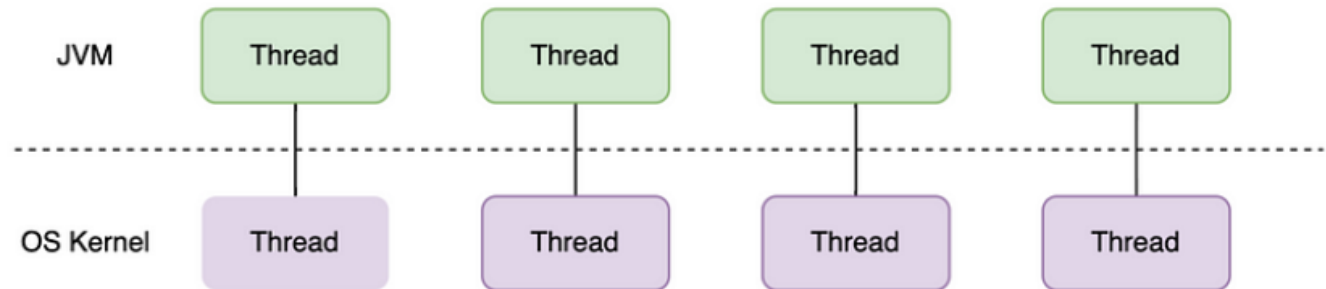
- worker threads which exist separately from the Runnable and Callable tasks they execute
- Minimize the overhead due to thread creation
- Methods for creating executors that use thread pools
 - `newFixedThreadPool` - fixed size of threads
 - Tasks processed sequentially (FIFO, LIFO, priority, etc.)
 - `newScheduledThreadPool` – fixed sized; supports delayed and periodic execution



Virtual Threads In Java 19

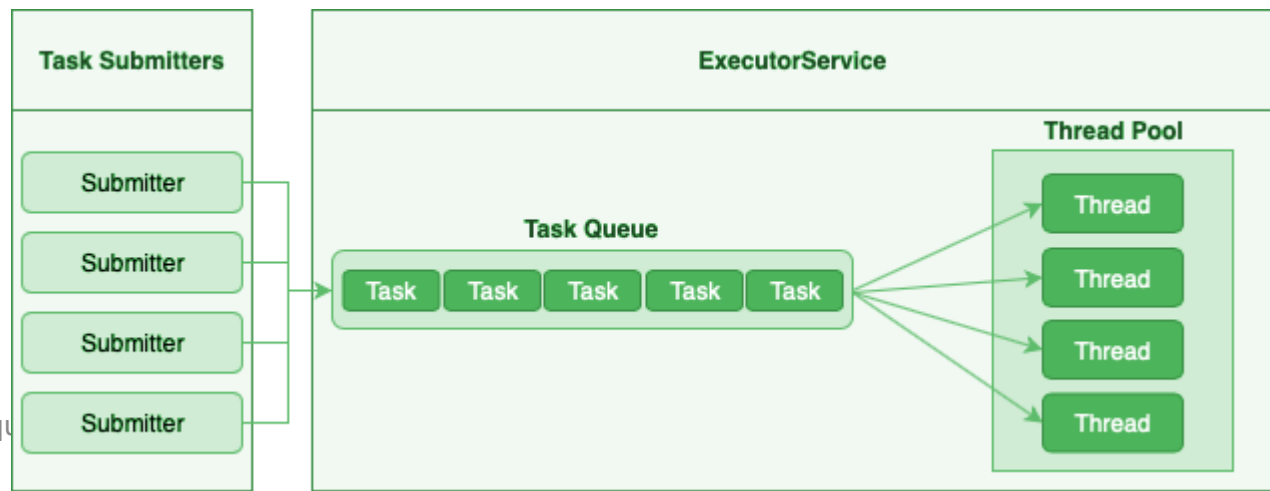
- **Java threads limitations**

- Threads of the machine are expensive, and the number is limited
 - Limiting factor long before other resources, such as CPU or network connections, are exhausted



Sources:
<https://medium.com/javarevisited/how-to-use-java-19-virtual-threads-c16a32bad5f7>

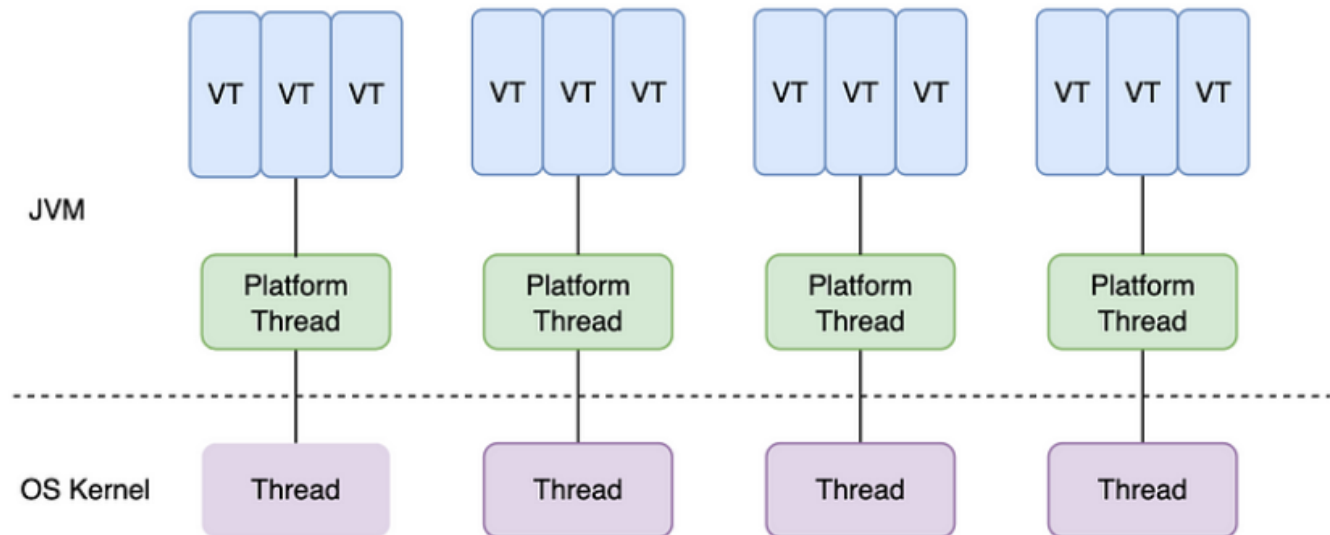
<https://stackoverflow.com/questions/65189805/java-thread-pool-task-execution-queueing>



Virtual Threads In Java 19

- **Virtual thread**

- Requires an OS thread to do CPU work, but doesn't hold the OS thread while waiting for other resources
- Have minimal overhead, so there can be many of them.
- Support thread-local variables, synchronized blocks, and thread interruption



Virtual Threads In Java 19

- **Virtual thread**

- Managed by the JVM, similarity by design
- Free of the system's context switch
- They don't block the carrier thread thus blocking is a much cheaper

`Thread.startVirtualThread(Runnable r) -replacement for calling thread.start()`

```
try {  
    Future future1 = Executors.newVirtualThreadPerTaskExecutor().submit(() -> fetchURL(url1));  
} catch (ExecutionException | InterruptedException e) {  
    response.fail(e);  
}
```

- Conventional threads - the application code is responsible for provisioning and dispensing OS resources.
- Virtual threads - JVM obtains and releases the resources from the operating system.
- The Java runtime arranges for it to run by *mounting* it on some platform thread, called a *carrier thread*.